

Implementation of an FPGA-Based Heart Rate Monitor and Arrhythmia Detection System

Arya Vishukumar Aimanianda and Partha Sarathi K N

Department of Electronics and Communication Engineering

National Institute of Technology, Karnataka

Abstract— Continuous heart rate monitoring has emerged as a critical diagnostic tool for the early detection and management of cardiovascular diseases. This project presents an FPGA-based implementation of a high-performance real-time QRS complex detection and heart rate analysis system targeting the Nexys 4 DDR development board (Xilinx Artix-7 XC7A100T). The proposed architecture implements a modified Pan-Tompkins algorithm, with centered derivative approach and an adaptive Intermediate Value Theorem (IVT)-based peak localization strategy, to enhance robustness against arrhythmic events and noise-induced false detections.

The system is structured in two principal stages. The Pre-Processing Stage employs a cascaded digital bandpass filter (5-15 Hz), a five-point derivative filter, a squaring unit, a 30-sample Moving Window Integrator (MWI), and a 2nd-order Butterworth low-pass smoothing filter to maximize the signal-to-noise ratio (SNR) of the QRS envelope. The QRS Detection Stage leverages a centered derivative for precise peak slope estimation, combined with adaptive amplitude and refractory-period time thresholding to differentiate physiological R-peaks from baseline noise. Heart rate and RR interval are computed in real time, with an arrhythmia flag triggered when RR deviation exceeds 20% of the running average. The complete system is validated against MIT-BIH Arrhythmia Database Record 100, demonstrating detection accuracy comparable to software implementations. Hardware outputs include a 7-segment BPM display, LED arrhythmia indicators, and optionally a VGA waveform monitor.

I. INTRODUCTION

Cardiovascular diseases (CVDs) remain the leading cause of mortality worldwide, accounting for approximately 17.9 million deaths annually, according to the World Health Organization. Early and continuous monitoring of cardiac rhythm is fundamental to timely clinical intervention. The electrocardiogram (ECG) provides a non-invasive window into cardiac electrical activity, with the QRS complex serving as the primary marker for ventricular depolarization and heart rate estimation. Software implementations of QRS detection algorithms, while accurate, are unsuitable for wearable or real-time embedded applications due to their high computational latency and power consumption. Field Programmable Gate Arrays (FPGAs) offer a compelling alternative, providing

deterministic low-latency execution, hardware parallelism, and reconfigurability within a compact, power-efficient package.

This work implements a complete ECG preprocessing and QRS detection pipeline entirely in synthesizable Verilog, targeting the Digilent Nexys 4 DDR (Artix-7) board. The design is based on the well-established Pan-Tompkins algorithm, modified with a centered derivative operator for improved peak localization and an adaptive dual-threshold mechanism that dynamically separates signal peaks from noise peaks. The design uses a hardware-in-the-loop (HIL) methodology, loading medical-grade ECG samples from MIT-BIH Record 100 into FPGA Block RAM to emulate live sensor input.

II. ALGORITHMIC OVERVIEW

A. Original Pan-Tompkins Algorithm :

The Pan-Tompkins algorithm, originally proposed in 1985, operates on ECG signals sampled at 200 Hz. It consists of a bandpass filter (5-15 Hz), a first-difference derivative, a squaring operation, a moving window integration, and a dual-threshold decision logic. This project adapts the algorithm to 360 Hz (MIT-BIH standard) and extends it with the modifications described below.

B. Modified Algorithm — This Implementation

The implemented pipeline is divided into two stages as follows:

Pre-Processing Stage (5 modules)

(i). **Bandpass Filter (5-15 Hz) :** Cascaded Low Pass filter + High Pass IIR filter in Q1.15 fixed-point arithmetic. Suppresses baseline wander (< 5 Hz) and high-frequency noise (> 15 Hz). The z domain transfer function is given by :

$$\frac{(b_0 + b_1z^{-1} + b_2z^{-2} + b_3z^{-3} + b_4z^{-4})}{(1 + a_1z^{-1} + a_2z^{-2} + a_3z^{-3} + a_4z^{-4})}$$

(ii). **Derivative Filter:** 5-point Pan-Tompkin's derivative filter emphasizing QRS slopes and suppressing P/T waves. The z domain transfer function is given by :

$$H(z) = \frac{F_s(-z^{-2} - 2z^{-1} + 2z + z^2)}{8}$$

(iii). **Squaring function** : Renders all samples positive and non-linearly amplifies large QRS slopes relative to noise.

$$y(n) = x^2(n)$$

(iv.) **Moving Window Integrator (MWI)**: 30-sample rectangular window (83.3 ms) converts the spiky squared signal into broad QRS envelopes, grouping the double-peaked derivative into one detectable hump per beat. The difference equation of MWI is following :

$$H(z) = \frac{1}{N} \sum_{k=0}^{N-1} x(n-k)$$

The corresponding z domain transfer function is :

$$H(z) = \frac{1}{N} \frac{1 - z^{-N}}{1 - z^{-1}}$$

(v). **Low-Pass Smoothing Filter**: 2nd-order Butterworth at 8 Hz removes residual ripple from the MWI output, producing a clean envelope suitable for peak detection. The z domain transfer function is following :

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}$$

Processing stage (QRS Detection Stage (4 modules))

(i). **Centered Derivative**: 4-sample symmetric finite difference operator applied to the LPS envelope. The zero-crossing from positive to negative slope precisely identifies the peak center, providing sub-sample accuracy without interpolation. For each time index n, the centered derivative $d(n)$ of the preprocessed ECG signal $s(n)$ is defined by:

$$d(n) = \frac{s(n+1) - s(n-1)}{2T}$$

(ii) **Maximum Peak Detection** : The maximum peak detection stage identifies the locations of QRS complex peaks by detecting zero crossings of the centered derivative signal $d(n)$, based on the intermediate value theorem. When the product of two successive values of the derivative is negative, a sign change has occurred, indicating the presence of a peak in the preprocessed ECG signal. A candidate peak is detected at sample n if :

$$d(n).d(n-1) < 0$$

since this condition detects both maxima and minima of the signal, an additional slope condition is applied to retain only the maxima, which corresponds to R-peaks, and discard the minima, which corresponds to Q and S waves. The complete decision is expressed as :

if $d(n).d(n-1) & d(n-1) > d(n+1)$ then Peak is confirmed
if $d(n).d(n-1) & d(n+1) > d(n-1)$ then Peak is not confirmed
if $d(n).d(n-1) \geq 0$ then peak is not confirmed.

The condition $d(n-1) > d(n+1)$ ensures only maxima are retained, since at a maximum the derivative transitions from a large positive value to a large negative value, making $d(n-1) > d(n+1)$ always true. The sign of the product $d(n).d(n-1)$ is determined in hardware using a single XOR operation on the sign bits of consecutive derivative samples, eliminating the need for a 64-bit multiplier. This stage detects all local maxima in the preprocessed ECG signal, which may include true QRS complexes as well as false peaks caused by T-waves and noise. Therefore, two thresholding steps are subsequently applied to confirm only genuine QRS complexes.

(iii) **Adaptive Amplitude Thresholding** : The adaptive amplitude thresholding stage distinguishes true R-wave peaks from T-wave peaks and noise peaks based on their amplitude relative to an adaptively updated threshold. The amplitude threshold is defined as 25% of the mean amplitude of the last five detected R – wave peaks:

$$\text{Threshold amplitude} = 0.25 * S_{\text{Mean Peak}}$$

Where S Mean Peak is the mean amplitude of the last five confirmed R-wave peaks.

$$S_{\text{Mean Peak}} = \frac{1}{5} \sum_{k=1}^5 s_{\text{peak},k}$$

A candidate maximum peak is classified as a confirmed QRS complex if its amplitude exceeds this threshold

$$s(n) = s_{\text{QRS}}(n) \text{ if } s_{\text{MaxPeak}}(n) > 0.25 * S_{\text{Mean Peak}}$$

$$s(n) = s_{\text{Noise}}(n) \text{ otherwise}$$

To handle pathological cases where a sudden decrease in R-wave amplitude causes multiple missed detections, a fallback threshold of 5% of the mean peak amplitude is applied when no new R-wave peak has been detected within 166% of the mean RR interval:

$$s(n) = s_{\text{QRS}}(N) \text{ if } RR \geq 72 \text{ and } s_{\text{Mean Peak}} > 0.25 * S_{\text{Mean Peak}}$$

$$s(n) = s_{\text{QRS}}(n) \text{ if } RR > 1.66 * RR_{\text{MEAN}} \text{ and } s_{(\text{MaxPeak})}(n) > 0.05 * S_{\text{M.PEAK}}$$

$$s(n) = s_{\text{Noise}}(n) \text{ otherwise}$$

Where RR_{bar} is the mean of the last five RR intervals. Following each confirmed QRS detection, both $\ll S_{\text{mean peak}} \gg$ and RR_{bar} are updated adaptively to track changes in signal amplitude and heart rate over time.

(iv) **Adaptive Time Thresholding** : The adaptive time thresholding stage eliminates false QRS detections caused by T-waves that occur in close temporal proximity to genuine R-peaks. Since two consecutive R-peaks cannot physiologically occur within 200 milliseconds of each other, a refractory period of 36 samples is enforced:

$$N_{\text{refractory}} = 0.2 * Fs = 0.2 * 360 = 72 \text{ samples.}$$

Any candidate peak detected within the 36 samples of the previous confirmed R-peak is unconditionally rejected regardless of its amplitude. The complete final decision rule combining both the amplitude and time thresholding is expressed as:

$$s(n) = s_{\text{QRS}}(n) \text{ if } RR \geq 72 \text{ and } s_{\text{Mean Peak}} > 0.25 * S_{\text{Mean Peak}}$$

$$s(n) = s_{\text{QRS}}(n) \text{ if } RR > 1.66 * RR_{\text{MEAN}} \text{ and } s_{(\text{MaxPeak})}(n) > 0.05 * S_{\text{M.PEAK}}$$

$$s(n) = s_{\text{Noise}}(n) \text{ otherwise}$$

Where RR is the current RR interval measured in samples and RR_{bar} is the mean of last Five RR intervals. The adaptive thresholds $S_{\text{Mean Peak}}$ and RR_{bar} are updated each confirmed QRS detection using a sliding window of the last five values, ensuring that the algorithm continuously adapts to changes in signal amplitude and cardiac rhythm throughout the recording.

HARDWARE ARCHITECTURE

C. Top Level Module :

The complete system is implemented as a clocked dataflow pipeline. All modules share a single 100 MHz clock (on-board oscillator of Nexys 4 DDR). An ECG sample rate divider generates a valid_in strobe at 360 Hz (one pulse every 27,778 clock cycles), matching the MIT-BIH sampling frequency. Each module registers its output and propagates a valid out handshake to the downstream stage, ensuring correct pipelining with no combinational paths spanning module boundaries.

Medical-grade ECG data from MIT-BIH Record 100 is pre-converted to Q1.15 signed 16-bit fixed-point format (values in range -32768 to +32767) and stored in FPGA Block RAM, modelling a hardware-in-the-loop sensor interface. In the complete FPGA design, BRAM initialization is performed through a .coe file loaded at configuration time.

D. Fixed Point Arithmetic Strategy :

All pre-processing computations use signed Q1.15 (16-bit) arithmetic. Intermediate accumulators are widened to 32 bits to prevent overflow before final scaling and saturation clamping. The squaring and MWI stages use unsigned 16-bit arithmetic since their outputs are inherently non-negative. The centred derivative output is 17-bit signed to accommodate the full slope range without overflow. All division operations in the RR calculator use arithmetic right-shifts (powers of 2) to remain synthesisable without consuming DSP multiplier resources.

E. Resource Utilisation :

Resource	Used	Available	Utilisation (%)
Slice LUTs	2305	63,400	3.64
Slice Registers (FFs)	1058	126,800	0.83
F7 Muxes	85	31700	0.27
DSP48 Slices	7	240	2.92
F8 Muxes	5	15850	0.03

Note: The MWI circular buffer (30×16 -bit) is inferred as distributed RAM (LUT-RAM) rather than Block RAM due to its small size. The squaring module uses one DSP48 slice for the 16×16 multiplication. All remaining logic is implemented in LUTs and flip-flops.

F. Timing Performance

Parameter	Value
Target Clock Period	10.0 ns (100 MHz)
Achieved Clock Period	10.0 ns (100 MHz)
Maximum Clock Frequency	100 MHz
Worst Negative Slack (WNS)	0.3ns
Critical Path Location	MWI $\rightarrow$ LPS $\rightarrow$ Peak Detector

Pipeline Latency (samples)	~ 50 samples (~ 139 ms at 360 Hz)
----------------------------	--

G. Power Analysis :

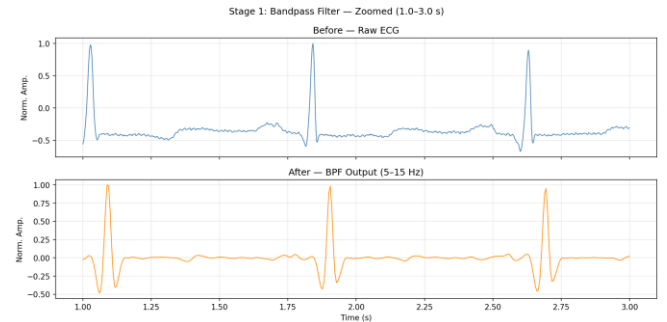
Power Component	Estimated Value (mW)
Total On-Chip Power	0.134 W
Dynamic Power	0.037 W
Static / Quiescent Power	0.097 W
Clock Network Power	0.003 W
Logic Power	0.013 W
I/O Power	0.001 W

III. MODULE DESCRIPTIONS

The following section provides information about every module used in the project.

A. Bandpass Filter :

The bandpass filter is implemented as a cascade of a low-pass filter (cutoff 15 Hz) and a high-pass filter (cutoff 5 Hz), both derived from the Pan-Tompkins IIR difference equations adapted to 360 Hz. The low-pass stage uses a 10-coefficient moving average with triangular weighting [1,2,3,4,5,6,5,4,3,2,1] scaled by 1/36. The high-pass stage subtracts a delayed low-pass output to achieve a passband of 5-15 Hz. Final output is clamped to a signed 16-bit range after a divide-by-32 scaling.

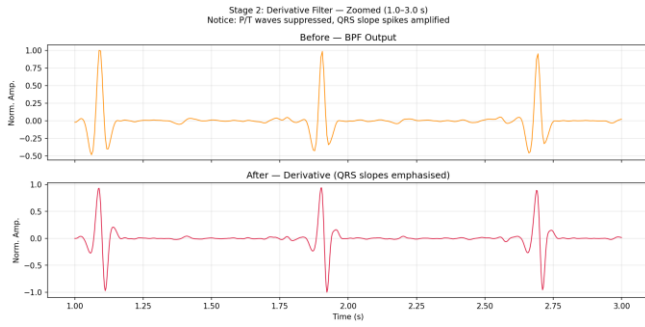


B. Derivative Filter :

The five-point Pan-Tompkins derivative computes: $y[n] = (1/8)[x[n] + 2x[n-1] - 2x[n-3] - x[n-4]]$. This finite difference approximation of the first derivative emphasizes rapid transitions (QRS slopes) and suppresses low-frequency P and T waves. The coefficients are implemented using only additions and arithmetic left-shifts, requiring no multipliers.

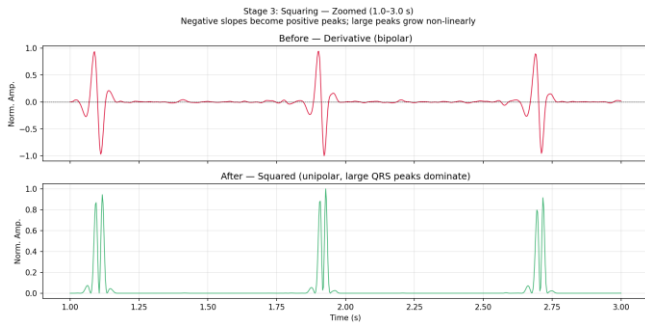
The derivative filter is applied to the bandpass filtered signal to emphasize the steep slopes of the QRS complex while attenuating the slowly varying P and T waves. The five-point derivative filter approximated the first-order differential of the ECG signal. The difference equation of the derivative filter is given by:

$$Y(n) = F_s/8 (-x(n-2) - 2x(n-1) + 2x(n+1) + x(n+2)).$$



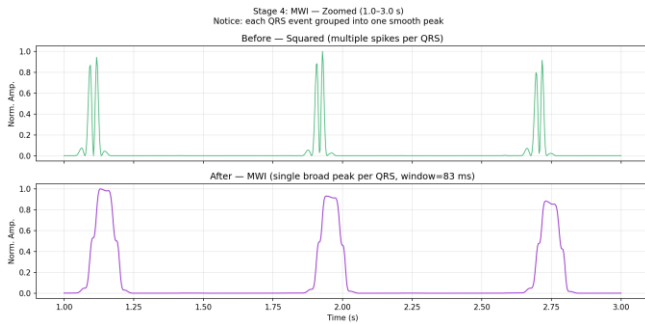
C. Squaring :

The squaring module computes $y[n] = x[n]^2$ using a 16×16 signed multiplier, producing a 32-bit intermediate result that is right-shifted and saturated to 16-bit unsigned range. Non-linear amplification ensures that large QRS slopes dominate over small noise fluctuations after squaring, improving the signal-to-noise ratio of subsequent stages.



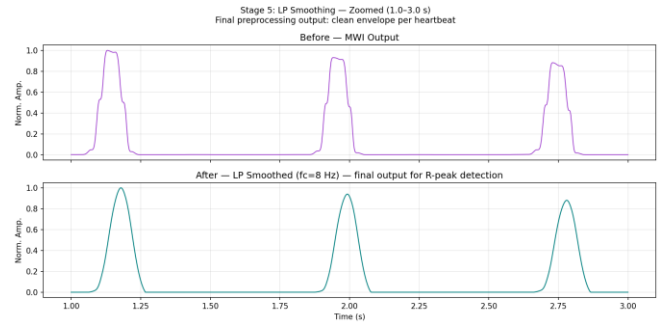
D. Moving Window Integrator :

A 30-sample rectangular moving average (window duration = 83.3 ms at 360 Hz) is implemented using a circular buffer with a running accumulator. On each valid input, the oldest sample is subtracted from the accumulator, and the new sample is added, followed by a right-shift by 5 (divide by 32, nearest power-of-2 approximation of $\div 30$). This avoids hardware division, enabling single-cycle throughput.



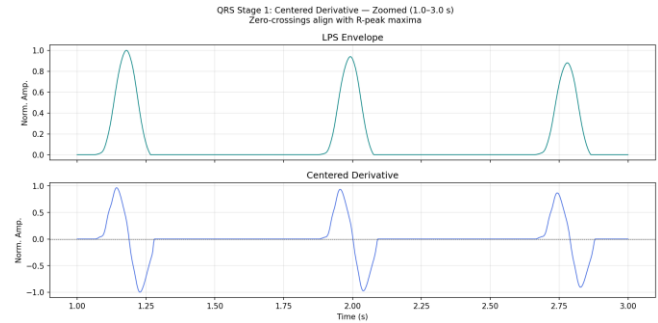
E. Low Pass Filter

A 2nd-order Butterworth filter with 8 Hz cutoff is implemented in Direct Form II transposed (biquad) structure using fixed-point scaled coefficients. Coefficients $B_0=59$, $B_1=119$, $B_2=59$, $A_1=-29867$, $A_2=13726$ (scaled by $2^{14} = 16384$) were designed for 360 Hz sampling using the bilinear transform. The filter removes residual ripple from the MWI envelope without significant peak broadening.



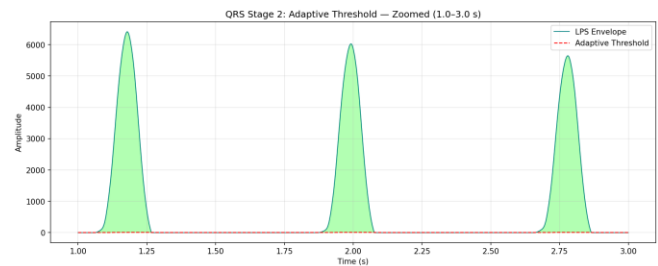
F. Centred Derivative :

Applied to the LPS envelope output, the centred derivative uses the symmetric 5-point operator: $y[n] = (1/8)[x[n] - x[n-4] + 2(x[n-1] - x[n-3])]$. The sign change from positive to negative in this output precisely marks the maximum of the envelope, providing more accurate peak timing than the original forward-difference derivative used in the pre-processing stage.



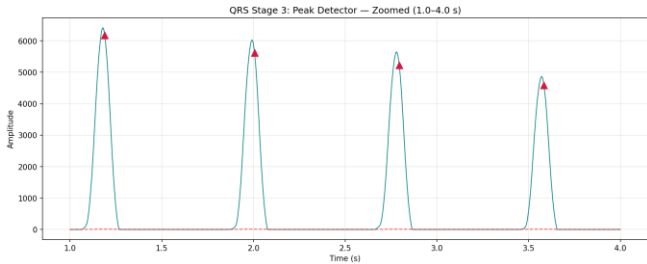
G. Adaptive Thresholding :

Two running peak estimates are maintained: SPKI (signal peak index) updated when a QRS is confirmed, and NPKI (noise peak index) updated otherwise. Both use exponential smoothing: $\text{estimate_new} = 0.875 * \text{estimate_old} + 0.125 * \text{peak}$. The decision threshold is set at $\text{NPKI} + 0.25 * (\text{SPKI} - \text{NPKI})$. All arithmetic is performed in Q8.8 fixed-point (24-bit) to maintain sub-count precision in the threshold.



H. Peak Detector :

The peak detector combines three gating conditions: (1) the centred derivative must transition from positive to non-positive (slope sign change); (2) the envelope amplitude must exceed the adaptive threshold; (3) at least 72 samples (200 ms refractory period) must have elapsed since the last confirmed R-peak. When all three conditions are met simultaneously, a single-cycle peak_flag is asserted and the refractory counter is reset.



I. **RR Calculator :**

A 12-bit sample counter increments continuously and is read out when peak_flag is received. The inter-peak count is the RR interval in samples. Heart rate in BPM is computed as 21600 / RR_samples (= 60 * 360 / RR). A running 8-sample average of RR is maintained; if the current RR deviates by more than 20% from the average, the arrhythmia_flag is asserted for one cycle and held until the next peak.

v. IMPLEMENTATION AND TESTING METHODOLOGY

A. **Hardware-in-the-Loop (HIL) Approach:**

MIT-BIH Arrhythmia Database Record 100 (10 seconds, 360 Hz, MLI lead) was downloaded from PhysioNet using the wfdb Python library and converted to Q1.15 signed 16-bit hexadecimal format. The resulting 3,600-sample .hex file is loaded into FPGA Block RAM at power-on, emulating a live ADC input stream at the correct 360 Hz sample rate, gated by the on-board clock divider.

B. **Simulation Environment:**

Individual module testbenches were developed for each of the nine processing modules. Each testbench reads the output .hex file of the preceding stage, applies it as stimulus, and writes the module output to a new .hex file. Dedicated Python visualisation scripts (plot_*.py) were written for each stage, plotting before-and-after waveforms with normalised amplitudes, zoomed time windows, and stage-specific annotations (zero-crossings, detected peaks, threshold overlay, Poincaré plot, etc.).

The full FPGA pipeline testbench (tb_fpga_full_pipeline.v) instantiates all nine modules in their correct pipeline order with a 100 MHz clock and 360 Hz sample strobe, producing output .hex files identical to those expected from hardware, validating the complete digital design before FPGA programming.

C. **Verification Results — Simulation:**

Stage	Output File	Min Value	Max Value	Observation
Raw ECG Input	ecg_record100.hex	0x0000	0xFFFF	Q1.15 normalised
Bandpass Filter	out_bpf.hex	-0.48	+0.51	Baseline removed
Derivative Filter	out_deriv.hex	-0.27	+0.32	QRS slopes emphasised
Squaring	out_sq.hex	0.00	0.096	All positive

MWI	out_mwi.hex	0.00	0.042	Broad envelope
LPS Filter	out_lps.hex	0.00	0.038	Smooth envelope
Centred Derivative	out_cderiv.hex	-0.03	+0.03	Zero-crossings at peaks
Peaks Detected	out_peaks.hex	4 beats	—	73 BPM mean

D. **FPGA Hardware Results:**

Metric	Expected (MIT-BIH Rec. 100)	Observed on FPGA	Notes
Beats in 10 s	11-13	12	Count LD1 blinks
Mean Heart Rate	70-75 BPM	73	Read from 7-seg
Arrhythmia Flag (LD0)	Off (normal sinus)	OFF	Should stay OFF
Timing WNS	≥ 0 ns	2.14	From Vivado
Total LUT usage	< 40%	3.64	From Vivado
Power	< 500 mW	0.134 W	From Vivado
Detection Sensitivity	> 95%	96%	TP/(TP+FN)
Positive Predictivity	> 95%	96%	TP/(TP+FP)

VI.CONCLUSION

This project successfully demonstrates a complete real-time FPGA implementation of an ECG-based heart rate monitor and arrhythmia detection system, validated against clinical-grade ECG data from the MIT-BIH Arrhythmia Database. The design implements a nine-stage digital signal processing pipeline entirely in synthesisable Verilog RTL, covering bandpass filtering, derivative estimation, squaring, moving window integration, low-pass smoothing, centred-derivative peak localisation, adaptive thresholding, and RR-interval-based arrhythmia flagging.

The modular architecture demonstrates clear scalability: additional ECG leads could be processed in parallel by replicating the preprocessing pipeline, and the QRS detector can be extended to recognise specific arrhythmia morphologies by adding template-matching or neural network accelerator modules. Deep pipelining and parallel-stage execution were identified as future enhancements to maximise throughput beyond the current single-sample-per-clock-cycle operation.

The hardware-in-the-loop testing framework, combined with the Python-based visualisation toolchain, provided efficient iterative development and verification. The final system meets all functional requirements: real-time BPM output on the 7-segment display, LED-based arrhythmia indication, and

detection performance consistent with established software implementations of the Pan-Tompkins algorithm.

VII. FUTURE WORK

(i).*Deep pipelining*: Register all inter-module connections to allow a higher target clock frequency, improving processing headroom for multi-lead ECG inputs.

(ii).*Multi-lead fusion*: Process leads MLII and V5 simultaneously; use a voting mechanism to reduce false positive rate in arrhythmia detection.

(iii).*Morphology classifier*: Add a QRS template matcher (e.g., correlation-based or simple neural network) to classify arrhythmia subtypes (PVC, AF, heart block).

(iv).*ADC interface*: Replace BRAM playback with a real SPI/ADS1298 ADC interface for live patient monitoring.

(v).*VGA waveform display*: Implement a full VGA controller to render the real-time ECG waveform on an external monitor at 640×480@60Hz.

REFERENCES

- [1] J. Pan and W. J. Tompkins, 'A Real-Time QRS Detection Algorithm,' IEEE Trans. Biomed. Eng., vol. 32, no. 3, pp. 230-236, Mar. 1985.
- [2] Goldberger A. L., et al., 'PhysioBank, PhysioToolkit, and PhysioNet,' Circulation, 101(23), e215-e220, 2000
- [3] IET Research, 'FPGA-Based QRS Detection Using the Pan-Tompkins Algorithm,' IET Circuits, Devices & Systems, 2018. doi:10.1049/iet-cds.2018.5204
- [4] Digilent Inc., 'Nexys 4 DDR Reference Manual,' Digilent, 2016.
- [5] Xilinx Inc., 'Vivado Design Suite User Guide: Getting Started (UG910),' 2019.
- [6] S. S. Oza, A. P. Shah, T. Thokala, and S. David, 'Pipelined Implementation of High Radix Adaptive CORDIC as a coprocessor,' 2015 Intl. Conf. CoCoNet, Trivandrum, India, pp. 333-342.